

1 - Job Search, Maps & Emails

Alright guys, we're almost there. We're going to do three things in this section. First, we'll add the job search functionality. We'll make the form that is in the Hero it's own search component. We'll also show that component on the jobs page. We'll have a route to submit to that will filter the jobs by keyword and location.

Next, we'll be implementing the Mapbox library to show a simple map on the details page with a marker for the location. We need to use geocoding to do this, which Mapbox provides. We do have the issue of the Mapbox API key. Initially, we'll add it to the JavaScript on the client, but that isn't the best thing to do because users can see that. So we'll create a geocode controller in Laravel with a route that we can hit from the client so that the API key is stored on the server.

Finally, we will set up emails so that when a job is applied to, the owner will get an email telling them. The email will also have the data and a download attachment for the resume. We will be using a service called Mailtrap for this. You don't have to pay anything because they have a generous free tier.

2 - Search Component & Route

We have a hero component now that has a form in it. I want to make that form a search component that can be reused in other parts of the application.

Let's create the component:

```
php artisan make:component Search
```

Open the `/resources/views/components/search.blade.php` and copy the form from the hero and paste it in the new file.

```
<form class="block mx-5 md:mx-auto md:space-x-2">
  <input
    type="text"
    name="keywords"
    placeholder="Keywords"
    class="w-full md:w-72 px-4 py-3 focus:outline-none"
  />
  <input
    type="text"
    name="location"
    placeholder="Location"
    class="w-full md:w-72 px-4 py-3 focus:outline-none"
  />
  <button
    class="w-full md:w-auto bg-blue-700 hover:bg-blue-600 text-white px-4 py-3
    focus:outline-none"
  >
    <i class="fa fa-search mr-1"></i> Search
```

```

    </button>
  </form>

```

Then in the hero component, replace the form with the search component.

```

@props(['title' => 'Find Your Dream Job'])
<section
  class="hero relative bg-cover bg-center bg-no-repeat h-80 flex items-center "
>
  <div class="overlay"></div>
  <div class="container mx-auto text-center z-10">
    <h2 class="text-4xl md:text-5xl text-white font-bold mb-8">{{ $title }}</h2>
    <x-search />
  </div>
</section>

```

I also want the search on the /jobs page. Open the `resources/views/jobs/index.blade.php` file and add the search component and a wrapper div at the top of the page as the first element in the `<x-layout>` component.

```

<div
  class="bg-blue-900 h-24 px-4 mb-4 flex justify-center items-center rounded"
>
  <x-search />
</div>

```

You should now see the search component on the home page and the jobs page.

Let's add an action to the form. Open the `/resources/views/components/search.blade.php` file and add the `action` attribute to the form tag. We are also using a method of GET.

```

<form
  class="block mx-5 md:mx-auto md:space-x-2"
  method="GET"
  action="{{ route('jobs.search') }}"
></form>

```

The page will show an error because the route does not exist. Let's create the route. Open the `routes/web.php` file and add the following route.

```
Route::get('/jobs/search', [JobController::class, 'search'])->name('jobs.search');
```

Add it above the job resource routes.

In the next lesson, we will add the controller method.

3 - Search Functionality

Now we need to create the `search` method in the `JobController`. Open the `app/Http/Controllers/JobController.php` file and add the following method.

```
// @desc Search for jobs
// @route GET /jobs/search
public function search(Request $request)
{
    $keywords = strtolower($request->input('keywords'));
    $location = strtolower($request->input('location'));

    $query = Job::query();

    if ($keywords) {
        $query->where(function ($q) use ($keywords) {
            $q->whereRaw('LOWER(title) like ?', ['%' . $keywords . '%'])
                ->orWhereRaw('LOWER(description) like ?', ['%' . $keywords .
                '%']);
        });
    }

    if ($location) {
        $query->where(function ($q) use ($location) {
            $q->whereRaw('LOWER(address) like ?', ['%' . $location . '%'])
                ->orWhereRaw('LOWER(city) like ?', ['%' . $location . '%'])
                ->orWhereRaw('LOWER(state) like ?', ['%' . $location . '%'])
                ->orWhereRaw('LOWER(zipcode) like ?', ['%' . $location . '%']);
        });
    }

    $jobs = $query->paginate(12);

    return view('jobs.index')->with('jobs', $jobs);
}
```

Some of this may be confusing, such as `$query->where(function ($q) use ($keywords)`.

This is where the main query logic happens. The `where()` method is being called on the query builder object (`$query`).

Inside the `where()` method, a closure (anonymous function) is used. This closure allows you to encapsulate multiple conditions for the where clause.

`use ($keywords)` is used to pass variables from the parent scope (in this case, `$keywords`) into the closure. Without `use`, the closure wouldn't have access to the `$keywords` variable since it's defined outside of the closure.

We are using the `whereRaw` method to pass in raw SQL so that we can use `LOWER` to make the search case-insensitive.

We are using the `paginate` method to paginate the search results. We are passing the search results to the regular `jobs.index` view.

That's it! Now the search form should work. Try searching for a job and see if the search results are displayed on the page.

Keep Text Values in Search Form

Let's add a hidden input to the search form to keep the text values in the search form when the page is refreshed. Open the `/resources/views/components/search.blade.php` file and add the following values to the inputs:

```
<input
  type="text"
  name="keywords"
  placeholder="Keywords"
  class="w-full md:w-72 px-4 py-3 focus:outline-none"
  value="{{ request('keywords') }}"
/>
<input
  type="text"
  name="location"
  placeholder="Location"
  class="w-full md:w-72 px-4 py-3 focus:outline-none"
  value="{{ request('location') }}"
/>
```

Back Button

Let's add a back button if showing search results. In the

`resources/views/jobs/index.blade.php` file, add the following code just below the div that wraps the search component:

```
<!-- Back Button -->
@if(request()->has('keywords') || request()->has('location'))
<a
  href="{{ route('jobs.index') }}"
  class="bg-gray-700 hover:bg-gray-600 text-white px-4 py-2 rounded mb-4 inline-block"
>
  <i class="fa fa-arrow-left mr-1"></i> Back
</a>
@endif
```

This will check for any search parameters in the URL and if there are any, it will display a back button.

4 - Mapbox Setup

Now I want to show a map with a marker on the job listing pages. We will use the address and other location fields in the database.

Create a Mapbox Account

First, you need to create a Mapbox account. Go to [Mapbox](https://account.mapbox.com/) and sign up for a free account. Go to <https://account.mapbox.com/> and copy your API key.

Open your `.env` file and add the following:

```
MAPBOX_API_KEY=YOUR_KEY
```

We are going to add a script tag directly into our show view. Open the `app/views/jobs/show.blade.php` file and add the following at the very bottom of the file:

```
<link
  href="https://api.mapbox.com/mapbox-gl-js/v2.7.0/mapbox-gl.css"
  rel="stylesheet"
/>
<script src="https://api.mapbox.com/mapbox-gl-js/v2.7.0/mapbox-gl.js"></script>
<script>
  document.addEventListener('DOMContentLoaded', function () {
    // Your Mapbox access token
    mapboxgl.accessToken = "{{ env('MAPBOX_API_KEY') }}";

    // Initialize the map
    const map = new mapboxgl.Map({
      container: 'map', // ID of the container element
      style: 'mapbox://styles/mapbox/streets-v11', // Map style
      center: [-74.5, 40], // Default center
      zoom: 9, // Default zoom level
    });

    // Get address from Laravel view
    const city = '{{ $job->city }}';
    const state = '{{ $job->state }}';
    const address = city + ', ' + state;

    // Geocode the address
    fetch(
      `https://api.mapbox.com/geocoding/v5/mapbox.places/${encodeURIComponent(
        address
      )}.json?access_token=${mapboxgl.accessToken}`
    )
```

```
.then((response) => response.json())
.then((data) => {
  if (data.features.length > 0) {
    const [longitude, latitude] = data.features[0].center;

    // Center the map and add a marker
    map.setCenter([longitude, latitude]);
    map.setZoom(14);

    new mapboxgl.Marker().setLngLat([longitude, latitude]).addTo(map);
  } else {
    console.error('No results found for the address.');
```

This script will initialize the map and add a marker to the location of the job. It will geocode the address and get the latitude and longitude to center the map and add a marker.

We need to add a tiny bit of CSS. Open the `public/css/style.css` file and add the following:

```
#applicant-form {
  position: relative;
  z-index: 2;
}

#map {
  z-index: 1;
  height: 400px;
}
```

This will give the map a height so it shows and it will make sure the applicant form is on top of the map.

The map should now show, which is good, however, your mapbox key is public and anyone can see it. We need to do a bit more work if you want to hide this from the public. We will do that next.

5 - Hide Mapbox Key

Right now our map is showing, but the key is visible. Let's hide it by using a proxy endpoint. We will create a new route that will call the Mapbox API and return the data to the client.

Create a Proxy Route

Let's create a new route that will call the Mapbox API and return the data to the client. Open the `routes/web.php` file and add the following import:

```
use App\Http\Controllers\GeocodeController;
```

Now add the following route:

```
Route::get('/geocode', [GeocodeController::class, 'geocode']);
```

Create a Controller

Now create a controller with the following command:

```
php artisan make:controller GeocodeController
```

Open the `app/Http/Controllers/GeocodeController.php` file and add the following import:

```
use Illuminate\Support\Facades\Http;
```

Add the following method:

```
public function geocode(Request $request): array
{
    $address = $request->input('address');
    $accessToken = env('MAPBOX_API_KEY');

    $response =
Http::get("https://api.mapbox.com/geocoding/v5/mapbox.places/{$address}.json", [
        'access_token' => $accessToken
    ]);

    return $response->json();
}
```

What we are going is taking in the address from the request, and then using the `Http` facade to make a GET request to the Mapbox API from the server side. We are passing in the address as a query parameter, and then passing in the access token as a query parameter. We are then returning the response as JSON. This way the key is only on the server side and not the client side.

Update the Blade File

Now let's update the script in the `/resources/views/jobs/show.blade.php` file. Replace the `fetch` call with the following:

```
// Call proxy endpoint
fetch(`/geocode?address=${encodeURIComponent(address)}`)
  .then(response => response.json())
  .then(data => {
    if (data.features.length > 0) {
      const [longitude, latitude] = data.features[0].center;

      // Center the map and add a marker
      map.setCenter([longitude, latitude]);
      map.setZoom(14);

      new mapboxgl.Marker()
        .setLngLat([longitude, latitude])
        .addTo(map);
    } else {
      console.error('No results found for the address.');
```

We are making a request to our own server from the frontend.

Test it out and you should see the map and marker.

6 - Sending Emails

We can send emails from Laravel. What I would like to do is notify the user when someone applies to their job. We can do this by sending an email. We're going to be using Mailtrap for this, which is a an emailing platform for testing and sending emails in production. We're going to use it for both. There's a very generous free tier that let's us send up to 200 emails per day. Obviously if you're building a production site then you'll want to look into the premium plans, but the free plan is more than enough for this project.

Since we're building locally, we can't just simply send emails from Laravel. Mailtrap gives us a sandbox smtp server to use in development. So we're going to get that setup now.

Set Up Mailtrap

Let's start by setting up Mailtrap. Go to [Mailtrap](#) and sign up for an account. Once you are signed up, click on "Start Testing" under "Email Testing". This will allow us to test emails in our local environment.

Once you do that, click on the "Add Inbox" button and give it a name. I will call mine "Workopia". You will then see your inbox. Click on that and where it says "Code Samples", click on PHP and choose "Laravel 9+" from the dropdown. This will give you the configuration that you need to add to your `.env` file. It will look something like this:

```
MAIL_MAILER=smtp
MAIL_HOST=sandbox.smtp.mailtrap.io
```



```
MAIL_PORT=2525
MAIL_USERNAME=kj5k3j54k3j45
MAIL_PASSWORD=*****er495
```

Up above click on the password and you will see an option to "Copy Password". Paste all of this in your `.env` file replacing the existing `MAIL_` variables.

I am also going to change the `MAIL_FROM_ADDRESS` to `noreply@workopia.dev`. I actually have the domain `workopia.dev`. I would suggest using a domain that you own.

Mailables

In Laravel, we can send emails using a Mailable. A Mailable is a class that contains the logic for sending an email. We can create a Mailable by running the following command:

```
php artisan make:mail JobApplied
```

Now open the `app/Mail/JobApplied.php` file. There are some methods such as `envelope`, `content` and `attachments`.

Within the `content` method, we can add the view that we want to use. Let's add the following:

```
public function content(): Content
{
    return new Content(
        view: 'emails.job-applied',
    );
}
```

For the subject, we can add the following:

```
public function envelope(): Envelope
{
    return new Envelope(
        subject: 'New Job Application',
    );
}
```

Create View

Let's create a view for our email. Create a file at `resources/views/emails/job-applied.blade.php`. In this file, we can add the HTML that we want to send in the email. For now, just add the following:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Workopia Job Application</title>
  </head>

  <body>
    <p>There has been a new job application to your Workopia listing</p>

    <p>Login to your Workopia account to view the application.</p>
  </body>
</html>
```

Send Email

Now that we have our Mailable set up, we can send the email. Let's send the email when someone applies to a job. Open the `ApplicantController` and

Add the following imports:

```
use App\Mail\JobApplied;
use Illuminate\Support\Facades\Mail;
```

Add the following code to the `store` method just above the redirect:

```
// Send email to owner
Mail::to($job->user->email)->send(new JobApplied());
```

Now try and apply for a job. Open your Mailtrap inbox and you should see the email that we just sent. It will have the job owner's email address as the recipient.

You may want to send data in the email. We will do that in the next lesson.

7 - Sending Data In Emails

We have the email being sent to the job owner. Let's add some data to the email. I want the job title and all the application data. We can do this by passing data to the Mailable class.

Open the `app/Http/controllers/ApplicantController.php` file and pass in the `$application` and `$job` variables to the Mailable class.

```
Mail::to($job->user->email)->send(new JobApplied($application, $job));
```

Update The Mailable Class

Open the `app/Mail/JobApplied.php` file and

Add the `$application` variable to the class.

```
class JobApplied extends Mailable {
    use Queueable, SerializesModels;

    public $application; // Add this line
    public $job; // Add this line

    //..
```

Update the constructor to accept the `$application` variable.

```
public function __construct($application, $job)
{
    $this->application = $application;
    $this->job = $job;
}
```

Update The View

Change the view to the following:

```
<!DOCTYPE html>
<html>

<head>
    <title>Workopia Job Application</title>
</head>

<body>
    <p>There has been a new job application to your Workopia listing</p>

    <p><strong>Job Title:</strong> {{ $job->title }}</p>

    <p><strong>Application Details:</strong></p>

    <p><strong>Full Name:</strong> {{ $application->full_name }}</p>
    <p><strong>Contact Number:</strong> {{ $application->contact_number }}</p>
    <p><strong>Contact Email:</strong> {{ $application->contact_email }}</p>
    <p><strong>Message:</strong> {{ $application->message }}</p>
    <p><strong>Location:</strong> {{ $application->location }}</p>

    <p>Log in to your Workopia account to view the application</p>
</body>
```

```
</html>
```

In the next lesson, we will send the resume pdf as an email attachment.

8 - Email Attachments

In this lesson, we will send the resume as an email attachment.

We are already passing the entire `$application` variable to the email, which includes the `resume_path` property. We can use this property to attach the resume to the email.

Open the `app/Mail/JobApplied.php` file and import the following:

```
use Illuminate\Mail\Mailables\Attachment;
```

Add the following to the `attachments` method:

```
public function attachments(): array
{
    // Attach the resume if it exists
    $attachments = [];
    if ($this->application->resume_path) {
        $attachments[] = Attachment::fromPath(storage_path('app/public/' . $this->application->resume_path))
            ->as($this->application->resume_path)
            ->withMime('application/pdf');
    }

    return $attachments;
}
```

We are using the `Attachment::fromPath` method to attach the resume. We are using the `storage_path` helper to get the path to the resume. We are using the `withMime` method to specify the MIME type of the attachment.

Now submit an application with a resume and it will be sent as an attachment.

9 - Setup Emails for Production

Now that we know our email is working, we need to setup our email for production. We can still use Mailtrap for this. Go to <https://mailtrap.io/home> and click on "Star Sending" under "Email API/SMTP". It will ask for the domain you want to use. I have the domain `workopia.dev` registered at Namecheap. Use a domain that you own.

You should see a screen like this:



Domain Setup

We need to add a few things in the domain registrar interface. I am using Namecheap so that is what this guide will be based on. If you are using a different registrar, the steps should be similar. There are instructions for other registrars on the right of this page.

Open Namecheap and go to the domain you want to use and click on "Manage".

Click on "Advanced DNS".

Click on "Add New Record".

Add the records that you see on the Mailtrap screen. For me, I added the following:

Domain Verification

- Type: CNAME
- Name/Host: 83u6tzdauplv3r1p
- Value: smtp.mailtrap.live
- TTL: Auto

DKIM Records

- Type: CNAME
- Name/Host: rwmt1._domainkey
- Value: rwmt1.dkim.smtp.mailtrap.live
- TTL: Auto
- Type: CNAME
- Name/Host: rwmt2._domainkey
- Value: rwmt2.dkim.smtp.mailtrap.live
- TTL: Auto

SPF Record

- Type: TXT
- Name/Host: @
- Value: v=spf1 include:spf.efwd.registrar-servers.com include:_spf.smtp.mailtrap.live ~all
- TTL: Auto DMARC Record

- Type: TXT
- Name/Host: _dmarc
- Value: v=DMARC1; p=none; rua=mailto:dmarc@smtp.mailtrap.live; ruf=mailto:dmarc@smtp.mailtrap.live; rf=afrf; pct=100
- TTL: Auto Domain Tracking
- Type: CNAME
- Name/Host: mt-link
- Value: t.mailtrap.live
- TTL: Auto

Now my Mailtrap page looks like this:



Everything is verified.

Update Laravel

Now click on the **Integrations** tab and click on PHP and then the "Laravel 9+" Dropdown. You will see the following:

```
MAIL_MAILER=smtp
MAIL_HOST=live.smtp.mailtrap.io
MAIL_PORT=587
MAIL_USERNAME=api
MAIL_PASSWORD=*****8eaa
```

Now your app is ready to send emails, however, if you try this now, you may get the following message:

```
Expected response code "250" but got code "550", with message "550 5.7.1 Security
check pending. Mailtrap is checking your domain credibility, it usually takes one
business day.". (code: 550)
```

You may see a box in the Mailtrap interface that says "Complete Compliance Check". In order to use Mailtrap in production, you need to complete the compliance check. Click on the box and fill out the form. You will need to provide your domain, company name, and a few other details. Once you submit, it will take a day or so to get approved and you can start sending emails.